TEXORA · LECTURE SHEET

Computer Software

System software against application software, the operating system and its management functions in detail, types of operating system, language translators with their translation flow, utilities, application categories with textile examples, levels of programming language, and open-source against proprietary software.

CSE-2105 · FUNDAMENTALS OF COMPUTING · TOPIC 3 · LEVEL 2 TERM 1

1. What is software?

Software is the set of instructions, programs, and associated data that directs a computer to perform tasks. Hardware is the physical equipment that can be seen and touched; software is intangible — it exists as stored patterns, and it is what makes the same hardware able to act, at different moments, as a typewriter, a calculator, a design studio, or a loom controller. Without software, hardware is inert; without hardware, software has nothing to run on. A collection of programs written for one purpose, together with its documentation, is often called a software package or an application.

Software divides into two great classes by purpose. **System software** manages the computer itself and provides the platform on which other programs run. **Application software** performs tasks for the user — writing, calculating, designing, communicating. The division is functional, not a matter of quality: both are essential, and they meet at the operating system, the most important piece of system software on any machine.

2. System software and application software compared

FEATURE	SYSTEM SOFTWARE	APPLICATION SOFTWARE
Purpose	Controls and manages the hardware and provides a platform for other software	Performs a specific task for the end user
Examples	Operating systems, device drivers, language translators, utilities	Word processors, spreadsheets, browsers, CAD, media players
When it runs	Starts when the machine is switched on and runs continuously in the background	Runs only when the user starts it, and stops when the task ends

Can the machine work without it?	No — without an operating system the hardware cannot be used at all	Yes — the machine runs normally, but that particular task cannot be done
User interaction	Mostly indirect; users interact with it through settings and system tools	Direct and continuous; designed around the user interface
Language level	Often written in low-level languages or system-level languages such as C for speed and hardware access	Usually written in high-level languages
Dependency	Generally machine- or platform-dependent	Often available in versions for several platforms
Number installed	One operating system normally controls a machine	Many applications coexist on one machine

Device drivers deserve a mention between the two classes: each driver is system software that teaches the operating system how to operate one specific device — a printer, a scanner, a particular graphics card. Installing a new peripheral usually means installing its driver, after which every application can use the device without knowing its details.

3. The operating system: functions in detail

An **operating system (OS)** is system software that acts as an intermediary between the user and the hardware, managing all resources and providing the environment in which applications run. Windows, Linux, macOS, Android, and iOS are familiar examples. Its functions group into six management duties.

Process management. A running program is a process. The OS creates, schedules, and terminates processes, allocating the CPU among them so that many programs appear to run at once. On a single core this is achieved by switching the CPU between processes thousands of times per second (multitasking); the scheduler decides which process runs next and for how long, and the OS handles processes that must communicate or wait for each other.

Memory management. The OS decides which parts of RAM each process may use, loads programs into free space, protects one process from overwriting another, and frees memory when a process ends. When RAM is full, it may move inactive pages to disk (virtual memory), making the machine appear to have more memory than is physically installed — at the cost of disk speed.

File management. The OS organises data on storage devices as named files within folders or directories. It creates, reads, writes, renames, copies, and deletes files, keeps track of where each file

physically lies on the disk, controls access permissions, and presents the same simple file interface whatever the underlying device.

Device management. Through drivers and input-output queues, the OS coordinates all peripherals: it buffers fast CPU output for slow printers (spooling lets a document be “printed” instantly while the printer works through it gradually), handles interrupts from devices, and shares one device fairly among competing processes.

Security and access control. The OS authenticates users (passwords, biometrics), enforces permissions on files and resources, separates user accounts, and provides the hooks on which antivirus and firewall tools build. It also keeps an audit of significant events in system logs.

User interface. The OS provides the means by which people command the machine: a graphical user interface (GUI) of windows, icons, menus, and a pointer, or a command-line interface (CLI) of typed commands. The GUI dominates personal computing for its ease of learning; the CLI remains prized by administrators for precision, scripting, and remote work. Beyond these six, the OS also performs error detection, resource accounting, and the orderly booting of the machine itself.

4. Types of operating system

TYPE	HOW IT WORKS	TYPICAL USES
Batch	Similar jobs are collected into batches and executed one after another with no interaction during running; the operator loads the batch and collects the printed results later. Efficient for the machine, slow for any single user.	Historical payroll and billing systems; still echoed in overnight bulk processing
Time-sharing	Many users at terminals share one machine; the CPU gives each user a tiny time slice in rotation, so everyone feels they have the machine to themselves. The basis of interactive multi-user computing.	University and company mainframes and minicomputers; conceptually behind every multitasking desktop
Real-time	Guarantees a response within a strict deadline; correctness depends on timeliness as much as on the result. Hard real-time systems treat a missed deadline as failure; soft real-time systems tolerate occasional lateness.	Loom and process controllers, medical monitors, flight control, robotics
Distributed	Coordinates a collection of separate networked computers so they work as one system, sharing files,	Large web services, cluster and cloud platforms, network file systems

processing, and resources across machines; a further type beyond single-machine systems.

Modern general-purpose systems blend several of these characters: a desktop OS multitasks interactively like a time-sharing system, runs scheduled batch-like background jobs, and contains real-time elements in media handling. The types remain valuable as descriptions of the guarantee each design puts first: throughput, responsiveness, deadline, or scale.

5. Language translators

Programs written by people must be converted into the machine code the processor executes. The software that performs this conversion is a **language translator**, and there are three kinds.

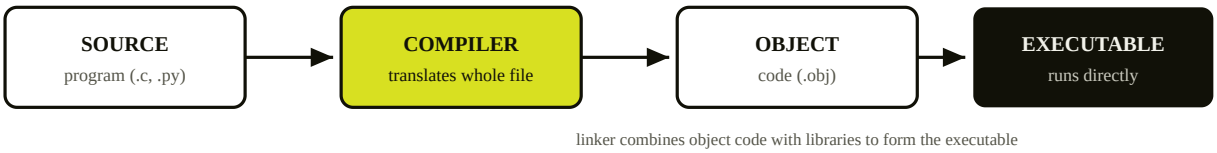


Figure 1 — Translation flow: source program, compiler, object code, then linking to an executable

FEATURE	ASSEMBLER	COMPILER	INTERPRETER
Input language	Assembly language (mnemonics)	High-level language, whole program	High-level language, line by line
Output	Machine (object) code	Object code, then an executable file after linking	No stored object code; executes directly as it translates
Translation time	One translation before running	One translation before running; can be slow for large programs	Translation repeats on every run, so execution is slower
Error reporting	Reports assembly errors before running	Reports errors for the whole program after compilation, before any execution	Stops at the first error it reaches during execution; later lines are unchecked
Typical languages	Assembly for a specific processor	C, C++, Fortran	Python, JavaScript (in their classic use), BASIC

In practical terms: a compiler trades a slower preparation step for a fast, stand-alone executable and early error detection; an interpreter trades execution speed for immediacy — the program runs at once

and is easy to test and modify. Many modern languages blur the line, compiling to an intermediate code that a runtime then interprets or compiles again at run time, but the distinction remains the correct first model.

6. Utility software

Utility software is system software that maintains, protects, and improves the working of the computer rather than running it or doing user tasks. Common utilities include antivirus and anti-malware scanners; disk cleanup and defragmentation tools; backup and recovery software; file compression tools that pack files into archives such as ZIP; disk formatting and partitioning tools; and system monitors that report CPU, memory, and temperature. Utilities are easy to overlook because they work in the background, but a machine without backup and protection utilities is a machine waiting to lose work.

7. Application software categories

CATEGORY	PURPOSE AND EXAMPLES	TEXTILE AND WORKPLACE USE
Word processing	Creating and editing documents: Microsoft Word, Google Docs, LibreOffice Writer	Production reports, specifications, correspondence
Spreadsheet	Tables of numbers with formulas, charts, and analysis: Microsoft Excel, Google Sheets	Consumption and costing calculations, production and quality data
Database management (DBMS)	Storing, organising, and querying structured records: MySQL, Microsoft Access, Oracle	Yarn and fabric stock records, order and buyer databases
CAD software	Precision design and drafting: AutoCAD, textile CAD suites	Woven and printed fabric design, garment pattern making and grading
Web browsers	Accessing the web: Chrome, Firefox, Edge	Buyer communication portals, research, cloud ERP access
Presentation	Slide-based communication: PowerPoint, Google Slides	Buyer presentations, training material
ERP software	Integrates production, inventory, accounts, and human resources in one system	End-to-end mill management from yarn purchase to shipment

Applications may be general-purpose, like the first five rows, usable in any industry, or specialised, like textile CAD and mill ERP, built for one industry's exact workflow. Choosing between them is an economic decision: general tools are cheap and flexible, specialised tools cost more but match the work precisely.

8. Levels of programming language

Programming languages are grouped by their distance from the hardware. The same instruction — add two numbers — shows the difference clearly.

LEVEL	CHARACTER	ONE-LINE EXAMPLE
Machine language (1st generation)	Binary instructions the processor executes directly. Machine-dependent, extremely tedious and error-prone to write, but needs no translation.	`10110000 01100001` — an instruction in pure binary (exact codes differ by processor)
Assembly language (2nd generation)	Short mnemonics stand for machine operations, with names for memory locations. Still machine-dependent and one instruction per machine step, but readable by a trained programmer; translated by an assembler.	`ADD A, 61H`
High-level language (3rd generation and beyond)	English-like statements, one statement doing the work of many machine instructions. Machine-independent, far faster to write and check; translated by a compiler or interpreter. Examples: C, Python, Java.	`total = a + 97`

Development has continued beyond the third generation: fourth-generation languages (query and report languages such as SQL) state what result is wanted rather than the steps to get it, and fifth-generation work explores logic and constraint-based programming for artificial intelligence. For engineering practice the first three levels, and their translators, are the essential core.

9. Open-source and proprietary software

FEATURE	OPEN-SOURCE SOFTWARE	PROPRIETARY SOFTWARE
Source code	Published; anyone may study, modify, and redistribute it under its licence	Kept secret by the owner; only the compiled program is supplied
Cost	Usually free of licence fees; costs, if any, are for support and services	Usually bought or subscribed to; licence terms restrict use
Examples	Linux, LibreOffice, Firefox, Python, MySQL	Microsoft Windows, Microsoft Office, Adobe Photoshop
Support	Community forums and documentation, plus optional commercial support	Vendor support contracts and official help desks
Modification	Users may adapt the software to their own needs	Modification is forbidden; only the vendor can change the program

Neither model is universally better. Open-source software offers freedom, transparency, and low cost, and it powers most of the world's servers; proprietary software often offers polish, integration, and accountable vendor support. Many organisations run both side by side, choosing per task — an approach a textile firm might follow by pairing a commercial ERP with open-source office and database tools.

Glossary

TERM	MEANING
Operating system	System software that manages hardware resources and provides the platform for applications.
Driver	System software that lets the operating system control one specific hardware device.
Process	A program in execution, with its own memory and state, managed and scheduled by the OS.
Compiler	A translator that converts a whole high-level program into object code before execution.
Interpreter	A translator that converts and executes a high-level program line by line, with no stored object code.
Assembler	A translator that converts assembly language mnemonics into machine code.

Utility	System software for maintenance and protection, such as antivirus or backup tools.
GUI / CLI	Graphical user interface (windows, icons, pointer) and command-line interface (typed commands).
Open-source	Software whose source code is published and may be studied, modified, and redistributed under its licence.

Key points & summary

- Software is the intangible set of programs that directs hardware; it divides into system software, which runs the machine, and application software, which serves the user.
- The operating system manages processes, memory, files, devices, security, and the user interface — no application runs without it.
- Operating systems are described as batch, time-sharing, real-time, and distributed, according to the guarantee each design puts first.
- Translators convert human-written code to machine code: the assembler for assembly language, the compiler for whole-program translation, the interpreter for line-by-line execution.
- Utility software maintains and protects the system: antivirus, backup, compression, and disk tools.
- Application categories include word processing, spreadsheets, DBMS, CAD, browsers, and industry systems such as textile ERP and CAD.
- Languages rise from machine code through assembly to high-level languages; open-source and proprietary models differ in code access, cost, and who may modify the software.

Practice

Answer in your own words; brief model answers follow.

1. Give four differences between system software and application software, with one example of each.
2. Describe any three operating system management functions and give one concrete event in which each is visibly at work.
3. A factory controller must respond to a sensor within a fixed short deadline, every time. Which type of operating system is required, and why would a batch system be unsuitable?
4. Compare a compiler and an interpreter on translation timing, output, execution speed, and error reporting, and name one language typically associated with each.
5. Write the idea “add the value 40 to the contents of a location called count” as it would appear at the high-level of programming, and explain why it cannot be written the same way in machine

language.

6. A small weaving firm is choosing software: state one argument for open-source office software and one argument for a proprietary, vendor-supported ERP package.

Answer key

1. Any four from the comparison table, for example: system software manages the hardware while application software does a user task; system software runs from switch-on continuously while applications run on demand; the machine cannot work without system software but works without any given application; users interact directly with applications and mostly indirectly with system software. Examples: an operating system (system) and a spreadsheet (application).
2. Any three, for example: process management — several programs stay responsive together because the OS schedules CPU time among them; file management — saving a document places it in a named file whose disk location the OS tracks; device management — a document appears to print instantly because the OS spools it to disk while the slow printer works. Memory management (loading programs, virtual memory) and security (log-in and permissions) are equally valid.
3. A real-time operating system, because correctness there includes meeting the deadline; a late answer is a wrong answer. A batch system collects jobs and runs them later with no interaction during execution, so a response could be delayed arbitrarily — the opposite of the guarantee required.
4. A compiler translates the whole program before running, produces object code and an executable, runs fast once compiled, and reports errors across the whole program before execution — typical language: C. An interpreter translates line by line during execution, produces no stored object code, runs more slowly, and stops at the first error reached — typical language: Python.
5. High-level form, for example: `count = count + 40`. Machine language has no names, words, or symbols — only binary operation codes and binary addresses for one specific processor — so the same idea must be expressed as a pattern of bits that the hardware, not a human reader, can act on.
6. Open-source office software: no licence fees and the source may be inspected and adapted, which suits letters and spreadsheets for a small firm. Proprietary ERP: vendor support and accountability for a system that runs production and accounts, where a failure stops the business and professional help must be guaranteed.

Texora — course notes prepared from the official syllabus of B.Sc. in Textile Engineering · Made by Sakib